

Auditoría de Implementación: Historias de Usuario Técnicas - 1er Sprint

PB01: Emisión, validación y manejo de tokens JWT (login y registro)

Estado: **Implementado**

Archivos y Ubicaciones

- **Backend:** PasteleriaAPI/Program.cs (Líneas 29-49 y 64). Configuración del middleware JwtBearer.
- **Backend:** PasteleriaAPI/Controllers/AuthController.cs (Líneas 30-46 para Login, 76-92 para Registro). Generación del token.
- **Frontend:** PasteleriaMaui/Services/AuthService.cs (Líneas 36-43 para Login, 56-63 para Registro). Deserialización de respuesta.
- **Frontend:** PasteleriaMaui/AppSession.cs (Línea 8). Propiedad en memoria Token.
- **Frontend:** PasteleriaMaui/Views/InicioPage.xaml.cs (Líneas 78-83). Limpieza del token en el logout.

Interacción

Al realizar un inicio de sesión o registro exitoso, `AuthController` crea un `JwtSecurityToken` firmado que contiene claims (Id, Nombre, Rol) y lo retorna en un objeto `AuthResponse`. En el frontend, `AuthService` lee este token, lo guarda permanentemente en `SecureStorage` (para persistencia) y simultáneamente en `AppSession.Token` (caché en memoria para acceso rápido y síncrono sin bloquear la UI).

Código Principal

```
1 var tokenHandler = new System.IdentityModel.Tokens.Jwt.JwtSecurityTokenHandler();
2 var key = System.Text.Encoding.UTF8.GetBytes("P@steleriaS3cr3tK3y1234567890123");
3 var tokenDescriptor = new Microsoft.IdentityModel.Tokens.SecurityTokenDescriptor
4 {
5     Subject = new System.Security.Claims.ClaimsIdentity(new[]
6     {
7         new System.Security.Claims.Claim("id", user.Id.ToString()),
8         new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Name, user.
9         Nombre),
10        new System.Security.Claims.Claim(System.Security.Claims.ClaimTypes.Role, user.
11        Rol)
12    }),
13    Expires = DateTime.UtcNow.AddDays(7),
14    SigningCredentials = new Microsoft.IdentityModel.Tokens.SigningCredentials(new
15    Microsoft.IdentityModel.Tokens.SymmetricSecurityKey(key), Microsoft.IdentityModel.
16    Tokens.SecurityAlgorithms.HmacSha256Signature)
17 };
18 var token = tokenHandler.CreateToken(tokenDescriptor);
19 var tokenString = tokenHandler.WriteToken(token);
20 return Ok(new { Token = tokenString, Usuario = user });
```

PasteleriaAPI/Controllers/AuthController.cs (Fragmento de Login)

PB16: Control de acceso basado en roles (RBAC) y reglas de autorización en endpoints

Estado: **Implementado**

Archivos y Ubicaciones

- **Backend:** PasteleriaAPI/Controllers/PastelController.cs (Líneas 39, 53, 94, 103, 110, 117, etc.).
- **Frontend:** PasteleriaMaui/Services/PastelApiService.cs (Líneas 21-25) y TransaccionesService.cs (Líneas 22-26).

Interacción

Los servicios HTTP de MAUI interceptan la creación del `HttpClient` en su constructor, recuperando el token desde `AppSession.Token` (inyectado durante el login) y adhiriéndolo al `DefaultRequestHeaders.Authorization`. En el servidor, el atributo `[Authorize(Roles = ".admin")]` bloquea el acceso si el claim `Role` del JWT no coincide, retornando HTTP 403 (Forbidden).

Código Principal

```
1 public PastelApiService()  
2 {  
3     var handler = MauiProgram.GetHttpClientHandler();  
4     _httpClient = new HttpClient(handler);  
5  
6     var token = AppSession.Token;  
7     if (!string.IsNullOrEmpty(token))  
8     {  
9         _httpClient.DefaultRequestHeaders.Authorization = new System.Net.Http.Headers.  
10         AuthenticationHeaderValue("Bearer", token);  
11     }
```

PasteleriaMaui/Services/PastelApiService.cs (Inyección del Bearer Token)

PB07: CRUD de entidades base (bizcochos, rellenos, glaseados)

Estado: **Implementado**

Archivos y Ubicaciones

- **Backend:** PasteleriaAPI/Controllers/PastelController.cs (Líneas 94-192). Endpoints POST, PUT, DELETE para cada catálogo.
- **Frontend:** PasteleriaMaui/Services/PastelApiService.cs (Líneas 81-128). Operaciones CRUD hacia la API.
- **Frontend:** PasteleriaMaui/Views/CatalogosPage.xaml. UI de administración.

Interacción

Se emplean verbos HTTP convencionales. La UI llama a `PastelApiService`, quien manda peticiones protegidas. Entity Framework actualiza directamente la base de datos local de SQLite. Todos los endpoints de modificación están restringidos solo a administradores mediante RBAC.

PB02: Endpoint GET de productos y renderizado del catálogo

Estado: **Implementado**

Archivos y Ubicaciones

- **Backend:** `PasteleriaAPI/Controllers/PastelController.cs` (Líneas 68-80, endpoint `todospasteles`).
- **Frontend:** `PasteleriaMaui/ViewModels/ListaPastelesViewModel.cs` y `ListaPastelesPage.xaml`.

Interacción

Al abrir la aplicación como cliente, se despliega la pantalla `ListaPastelesPage`. El `ViewModel` ejecuta una petición asíncrona hacia el servidor, el cual retorna todos los pasteles estáticos de la base de datos (filtrados para excluir los pedidos personalizados).

PB03: Endpoint GET por ID y vista de detalles del producto

Estado: **Implementado**

Archivos y Ubicaciones

- **Backend:** `PasteleriaAPI/Controllers/PastelController.cs` (Líneas 22-34). Endpoint `[HttpGet("id")]`.
- **Frontend:** `PasteleriaMaui/Services/PastelApiService.cs` (Líneas 74-85). Método `GetPastelByIdAsync`.
- **Frontend:** `PasteleriaMaui/ViewModels/DetallePastelViewModel.cs` (Líneas 23-37). Consumo del endpoint.

Interacción

Cuando el usuario selecciona un pastel en el catálogo general de MAUI, el objeto básico pasa vía `QueryProperty` a la vista de detalle. Inmediatamente en el setter de la propiedad, si el objeto no tiene sus relaciones anidadas completas, se dispara `CargarDetallesCompletoAsync` que hace un HTTP GET por ID al backend. El backend usa `.Include()` en Entity Framework para recuperar la entidad anidada con `Categoria`, `Bizcocho`, `Relleno` y `Glaseado`, devolviéndolo al frontend para refrescar la UI (`Data Binding`).

Código Principal

```
1 [HttpGet("{id}")]
2 public async Task<ActionResult<Pastel>> GetPastelById(int id, [FromServices]
   ApplicationDbContext db)
3 {
4     var pastel = await db.Pasteles
5         .Include(p => p.Categoria)
6         .Include(p => p.Bizcocho)
7         .Include(p => p.Relleno)
8         .Include(p => p.Glaseado)
9         .FirstOrDefaultAsync(p => p.Id == id);
10 }
```

```
11     if (pastel == null) return NotFound();
12     return Ok(pastel);
13 }
```

PasteleriaAPI/Controllers/PastelController.cs (Endpoint GET por ID)

PB08: Endpoint POST para crear pasteles protegido por RBAC

Estado: **Implementado**

Archivos y Ubicaciones

- **Backend:** PasteleriaAPI/Controllers/PastelController.cs (Líneas 53-66). Endpoint [HttpPost("nuevopastel")].
- **Frontend:** PasteleriaMaui/ViewModels/RegistrarPastelViewModel.cs (Líneas 73-104). Validación local y petición al servidor.

Interacción

El usuario completa el formulario en `RegistrarPastelPage.xaml`. El `ViewModel` verifica localmente que no existan campos vacíos ni selecciones nulas. Al enviar, `PastelApiService` serializa el objeto `Pastel` a JSON e inyecta el Bearer Token del Administrador. El backend valida el token mediante el atributo `[Authorize(Roles = ".admin")]` y delega la lógica final al `PastelService` para inserción en base de datos.